

AWS Well-Architected Framework Review — Batch 2: ROCA Event Management Flows

1 Executive Summary

The Batch 2 Well-Architected review of ROCA's Event Creation and Event Discovery/RSVP flows reveals that all five security and architectural gaps identified in the Batch 1 Onboarding review remain unremediated — and three of them (missing WAF, missing GuardDuty, and JWT tokens lacking rotation) have escalated to P0-Critical severity now that these flows process payments through Stripe. With PCI DSS v4.0.1 fully mandatory since March 2025 [1, 2], ROCA's payment API endpoints are exposed to injection attacks without WAF [3], lack threat detection on Lambda invocations without GuardDuty [4, 5], and are vulnerable to token replay without refresh token rotation per RFC 9700 [6, 7]. These 12 engineering days of P0 remediation should begin immediately.

The highest reliability risk is RSVP race conditions under concurrent bookings, where multiple Lambda invocations can simultaneously claim the last available event spots. The recommended mitigation combines PostgreSQL `SELECT FOR UPDATE SKIP LOCKED` for atomic capacity decrements [8, 9] with Redis distributed pre-locks to serialize critical booking paths [10], fronted by RDS Proxy to prevent connection exhaustion during traffic spikes [11]. Waitlist promotion and event cancellation cascades — which must orchestrate RSVP reversals, Stripe refunds with idempotency keys [12], and notification delivery — should be implemented as Step Functions sagas with compensating transactions [13, 14], directly addressing the Batch 1 gap of state machines embedded in application code [15].

The total remediation roadmap spans approximately 73 engineering days across four priority tiers, but the most critical risk reduction is concentrated in the first 12 days (P0) covering WAF, GuardDuty, DLQ configuration, and JWT rotation, followed by 33 days of P1 architectural work on Step Functions migration, concurrency control, and EventBridge decoupling. Delaying P0 items exposes ROCA to both PCI DSS non-compliance and silent financial data loss from missing dead-letter queues on payment webhook processing [15, 16].

2 Introduction and Scope

This report presents the Batch 2 AWS Well-Architected Framework Review for the ROCA church management application, covering two critical flows: **Event Creation** (a 6-step wizard) and **Event Discovery and RSVP** (a 5-step user flow). The review evaluates all six Well-Architected pillars — Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, and Sustainability — against the current architecture and identifies risks specific to these flows.

ROCA's technology stack comprises AWS Lambda with Node.js Express behind API Gateway REST, PostgreSQL 15 on RDS Multi-AZ for persistence, Redis 7 on ElastiCache for caching, S3 and CloudFront for media delivery, SES and SNS for notifications, Stripe for payments and ticketing, Socket.io over WebSocket for real-time updates, and JWT with OAuth 2.0 for authentication. This stack is well-suited for a serverless event-driven architecture but introduces specific challenges around payment compliance, concurrency control, and recurring event management that this review addresses.

Critically, this Batch 2 review cross-references five gaps identified during the Batch 1 Onboarding flow review: state machines implemented in application code rather than Step Functions, JWT tokens lacking rotation, absence of dead-letter queues on SQS, missing WAF and GuardDuty deployments, and no EventBridge event bus. Several of these gaps carry elevated risk in the context of payment processing and high-concurrency RSVP booking, making their remediation more urgent than when first identified.

3 Flow Architecture and State Machines

Chart Explanation: This diagram illustrates the state transitions for both flows under review. The Event Creation flow progresses through DRAFT → PUBLISHED → ACTIVE → COMPLETED, with key risk points at RRULE parsing during draft creation, state machine orchestration at publication, and cancellation cascade effects at completion. The RSVP flow transitions through PENDING → CONFIRMED → CHECKED_IN → ATTENDED, with a WAITLISTED branch for capacity-exceeded scenarios. Risk callouts highlight race conditions during booking, PCI DSS compliance at payment, and search performance during discovery.

Event Creation Flow. The Event Creation wizard spans six steps: basic event details entry, RRULE recurring pattern configuration, Stripe ticketing and pricing setup, volunteer role assignment, media upload, and review/publish. The ROCA API materializes recurring event instances at series creation time via the `/event-series` POST endpoint, storing the RRULE pattern in an `event_series` table with an `exceptions` column and supporting edit scopes of `this_only`, `this_and_future`, or `all` [17]. State transitions from DRAFT through PUBLISHED, ACTIVE,

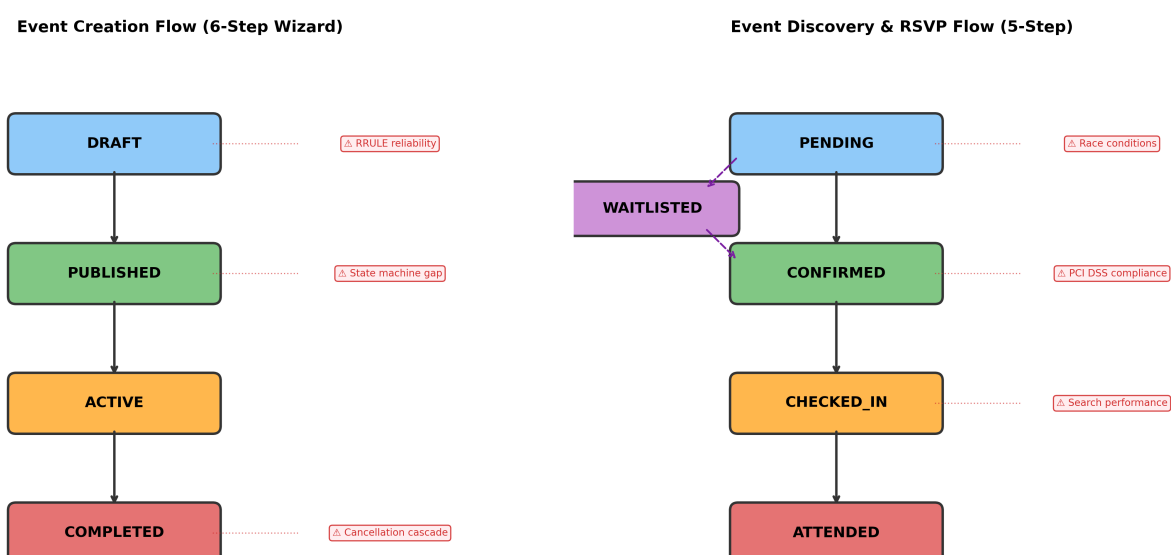


Figure 1: Event Creation and RSVP State Flows

and COMPLETED must be orchestrated reliably, with particular attention to the cascading effects when an event is cancelled after RSVPs and payments have been processed.

Event Discovery and RSVP Flow. The RSVP flow encompasses five steps: event browsing and search, event detail viewing, RSVP submission with capacity validation, Stripe checkout for paid events, and confirmation. RSVP states progress from PENDING (reservation initiated) to CONFIRMED (payment successful or free event accepted), CHECKED_IN (on-site verification), and ATTENDED (post-event). When capacity is exhausted, users are routed to a WAITLISTED state with automatic promotion when cancellations occur. The concurrency challenges in this flow — where multiple Lambda invocations may simultaneously attempt to claim the last available spots — represent the highest-risk area in this review.

4 Operational Excellence

State Machine Migration. The Batch 1 review identified that workflow state machines are implemented in application code rather than AWS Step Functions. This gap is amplified in Batch 2, where both the Event Creation wizard (with its multi-step validation, RRULE expansion, Stripe session creation, and volunteer assignment) and the RSVP flow (with capacity checks, payment processing, and confirmation) involve complex multi-service orchestration. AWS recommends using Step Functions to “minimize the amount of custom try/catch, back-off, and retry logic within your Serverless applications,” including using Step Functions integrations to save failed state runs into a DLQ [15]. Step Functions provides a fully managed service for orchestrating serverless workflows as visual state machines, coordinating AWS services like Lambda, DynamoDB, and SQS into reliable, repeatable processes [18]. Migrating both flows to Step Functions would provide built-in retry with exponential backoff, visual debugging through execution history, and automatic error routing to catch blocks [19].

Distributed Tracing and Observability. X-Ray should be enabled across Lambda functions, API Gateway, and the future Step Functions state machines to trace requests end-to-end. X-Ray traces Lambda functions, Step Functions, and API Gateway, helping visualize state machine components, identify performance bottlenecks, and troubleshoot errors [20]. The Serverless Lens recommends structured logging, distributed traces, and monitoring of metrics as core observability best practices, implementable via Lambda Powertools [21]. For ROCA, custom CloudWatch metrics should track event creation completion rates, RSVP conversion funnels, payment success/failure ratios, and waitlist promotion latency.

Dead-Letter Queue Configuration. The Batch 1 finding of missing DLQs on SQS queues is particularly dangerous for payment-related message flows. AWS recommends configuring Maximum Retry Attempts, Maximum Record Age, and Destination on Failure to control retries and remove poison-pill messages [15]. Best practice is to set maximum receives to 3 retries with a 4-day message retention period on the DLQ [22], allowing sufficient time for offline analysis or automated remediation of failed payment webhooks, notification deliveries, and RSVP state

transitions.

5 Security and PCI DSS Compliance

PCI DSS v4.0.1 Scope and Stripe Integration. PCI DSS v4.0.1 became the only active version as of March 31, 2025, with all 51 future-dated requirements now mandatory and no remaining best-practice carve-outs [1]. The PCI SSC confirmed 64 new requirements total, 51 of which were future-dated [2]. For ROCA, using Stripe Checkout (hosted payment page) qualifies the application for SAQ A, the simplest self-assessment questionnaire with approximately 25 questions, since card data never touches ROCA's servers [23]. However, using Stripe does not make ROCA fully PCI compliant — Stripe certifies its own systems, not the merchant's website [24]. Notably, SAQ A eligibility criteria tightened in January 2025: Requirements 6.4.3 and 11.6.1 were removed from the questionnaire and replaced with a new eligibility condition requiring merchants to confirm their payment service provider handles script security [25]. ROCA must verify this attestation is in place.

WAF and GuardDuty Deployment. The Batch 1 gap of missing WAF and GuardDuty is now a P0-Critical finding given the payment processing context. AWS WAF protects API Gateway REST APIs from common web exploits including SQL injection and cross-site scripting attacks [3], and AWS announced WAF support for PCI DSS 4.0 compliance protection with partner solutions in March 2025 [26]. GuardDuty Lambda Protection monitors Lambda network activity logs to identify potential security threats when Lambda functions are invoked [4], using ML-based anomaly detection to identify anomalous API invocations across attack tactics including discovery, initial access, persistence, privilege escalation, and data exfiltration [5]. Both services should be deployed immediately to protect payment endpoints and detect compromised Lambda functions.

JWT Token Rotation and Secrets Management. The Batch 1 gap of JWT tokens lacking rotation is critical for a payment-processing application. Access tokens should have lifetimes of 5 to 60 minutes, and refresh tokens should rotate on every use so that theft is detectable and automatically invalidates the entire token family [6]. RFC 9700 (OAuth 2.0 Security Best Current Practice) mandates that refresh tokens issued to public clients be rotated on every use with replay/reuse detection, or be sender-constrained [7]. Longer-lived user sessions should be handled through refresh tokens rather than by extending access token lifetimes [27]. For API Gateway, JWT authorizers validate tokens by checking kid, iss, aud, exp, nbf, iat, and scope claims, and API Gateway caches public keys for two hours — when rotating keys, a grace period during which both old and new keys are valid is recommended [28]. For church management SaaS applications specifically, JWT authorizers should be configured with audience restriction per tenant, authorization scopes to differentiate access levels (e.g., admin, pastor, member), and PKCE for browser-based OAuth flows per RFC 10017 [29].

Stripe API Key Rotation. Each Stripe API key should be stored as a separate secret in AWS Secrets Manager with structured JSON, and Secrets Manager supports rotation as frequently as every four hours via custom Lambda rotation functions [30]. Stripe has published an official guide on securing Stripe API keys in AWS with automatic rotation using Secrets Manager [31], which ROCA should implement to meet both PCI DSS key management requirements and organizational security policies.

6 Reliability and Concurrency Control

RSVP Race Condition Prevention. The highest-reliability risk in Batch 2 is concurrent RSVP bookings exhausting event capacity. The recommended pattern is PostgreSQL's `SELECT FOR UPDATE SKIP LOCKED`, which acquires exclusive row-level locks and skips already-locked rows rather than blocking. As documented, if two `SELECT FOR UPDATE` statements execute concurrently, one may wait for a long period, and in high-concurrency scenarios this leads to bottlenecks — `SKIP LOCKED` solves this by letting concurrent transactions process different available slots [8]. The atomic capacity decrement pattern — `UPDATE events SET capacity = capacity - 1 WHERE event_id = $1 AND capacity > 0 RETURNING *` — enforces capacity in a single statement, preventing overbooking without requiring a separate read [9].

Optimistic Concurrency and Advisory Lock Caveats. Optimistic concurrency control using a version column avoids lock overhead but requires retry logic on conflict and is generally used in environments with low data contention; when conflicts are frequent, the cost of repeatedly restarting transactions hurts performance significantly [32]. Critically, session-level advisory locks (`pg_advisory_lock`) must be avoided with RDS Proxy, as they cause connection pinning where RDS Proxy pins a client connection to a specific DB connection, reducing the effectiveness of connection reuse [33]. If advisory locks are needed, transaction-level `pg_advisory_xact_lock` should be used instead, as these release automatically at transaction end.

Redis Distributed Locks for Pre-Database Coordination. Redis distributed locks (`SET lock:event:{id} NX EX 5`) provide a fast coordination layer before hitting the database, throttling concurrent Lambda invocations to serialize critical booking paths [10]. For single-region deployments, a single Redis `SET NX EX` suffices; the Redlock

algorithm across N independent masters with a majority quorum is recommended for higher fault tolerance [34]. The TTL on Redis locks should always be set shorter than the Lambda timeout to prevent orphaned locks.

Waitlist Promotion Pattern. The recommended atomic waitlist promotion uses `SELECT FOR UPDATE SKIP LOCKED` within a transaction: first attempt the capacity decrement; if zero rows are returned, insert into the waitlist; on cancellation, atomically increment capacity and promote the next waitlisted entry via a `DELETE ... RETURNING *` with `FOR UPDATE SKIP LOCKED` ordering by creation timestamp. RDS Proxy is essential for Lambda to avoid exhausting database connections, as Lambda can easily scale to tens of thousands of concurrent requests while relational databases are not designed to accept the same number of concurrent connections [11].

Event Cancellation Saga. Event cancellation triggers cascading effects across RSVPs, payments, and notifications. AWS Prescriptive Guidance recommends Step Functions as the saga execution coordinator, where each microservice step has a corresponding compensating transaction that runs in reverse order on failure, ensuring data consistency without distributed ACID transactions [13]. The compensating transactions execute in reverse order (e.g., reverse notification → reverse refund → reverse RSVP update), with all paths converging to a final notification step [14]. Each Stripe refund request should include a unique idempotency key (up to 255 characters) to safely retry on network errors [12], and Stripe's Batch API supports files up to 5 GB with concurrency of 50–100 for independent operations [35]. For failed refund messages, SQS DLQs should be configured with a redrive policy setting `maxReceiveCount` (e.g., 5), so after that many failed processing attempts the message moves to the DLQ, and the dead-letter queue redrive feature can replay messages back to the source queue after fixing the consumer [16].

7 Performance Efficiency

Search Performance at Scale. Event Discovery relies on PostgreSQL full-text search, which should use GIN indexes on `tsvector` columns supporting the `@@` match operator [36]. However, GIN indexes have significant update overhead — every `INSERT`, `UPDATE`, or `DELETE` on indexed columns requires maintaining the GIN posting tree structure, and `VACUUM` operations can cause buffer contention on high-update tables [37]. For ROCA's read-heavy event discovery workload, GIN is the preferred choice, but a Redis cache-aside pattern should front the search queries: hash query parameters (search terms, filters, date ranges) to form a cache key, check Redis first, return cached data on a hit, or query PostgreSQL on a miss and store the result with a TTL [38]. Cache entries should be invalidated when events transition states (`DRAFT` → `PUBLISHED` → `ACTIVE`) using event-type-prefixed keys and pattern-based invalidation.

RRULE Parsing Reliability. The `rrule.js` library is the established Node.js option for RFC 5545 recurrence rules, supporting `RRuleSet` with `EXRULE`, `EXDATE`, and `RDATE` for exception handling [39]. However, `rrule-temporal` is a newer alternative built on the TC39 Temporal API, described as the first fully compliant RFC-5545 processing library with added RFC-7529 support for non-Gregorian calendars [40]. For DST edge cases, the Temporal API provides explicit timezone-aware arithmetic that avoids the ambiguity pitfalls of JavaScript's `Date` object, making `rrule-temporal` a stronger choice for production systems handling cross-timezone recurring events. When expanding long-running series (e.g., daily events over multiple years), ROCA should limit materialization windows to 6–12 months ahead and regenerate lazily to avoid CPU and memory spikes during Lambda execution.

Lambda Cold Start Mitigation. Lambda best practices recommend initializing SDK clients and database connections outside of the function handler, and caching static assets locally in the `/tmp` directory, so that subsequent invocations can reuse these resources, saving cost by reducing function run time [41]. For ROCA's payment-critical paths, provisioned concurrency should be considered for the RSVP checkout Lambda to eliminate cold start latency during peak booking periods.

8 Cost Optimization and Sustainability

Compute Cost Management. Lambda reserved concurrency incurs no additional charges while capping and guaranteeing function capacity [42], making it a zero-cost reliability improvement. For RDS and ElastiCache, Reserved Instances can save up to 72% over on-demand pricing with 1- or 3-year commitments across All Upfront, Partial Upfront, or No Upfront options [43]. Lambda memory should be right-sized using AWS Lambda Power Tuning, as memory can be configured between 128 MB and 10,240 MB in 1-MB increments with CPU allocated proportionally — at 1,769 MB a function has the equivalent of one vCPU [44]. CloudWatch metrics should be monitored to identify over-provisioned functions [45].

Invocation Efficiency and Event-Driven Architecture. Configuring event-source mapping batch window and batch size ensures functions are invoked only when sufficient messages accumulate, dramatically reducing invocation count — for example, batching 100 messages can reduce one million invocations to 10,000 [46]. The Batch 1 gap of no EventBridge event bus should be addressed by deploying a custom EventBridge bus where producers publish events (e.g., `event.cancelled`, `rsvp.confirmed`) and rules with content-based filtering deliver events

to independent targets without point-to-point coupling [47]. The Schema Registry provides a central collection of versioned schemas enabling teams to discover event structures and generate code bindings [48], which is essential as ROCA's event types grow across flows.

Storage and Transfer Optimization. S3 Intelligent-Tiering should be enabled for event media assets (flyers, images, videos) to automatically move infrequently accessed objects to lower-cost tiers. CloudFront caching for static media reduces S3 transfer costs and improves global delivery latency. ElastiCache data tiering, which uses lower-cost SSDs alongside memory, is suitable for workloads where only a fraction of the dataset is accessed regularly — appropriate for ROCA's event search cache where recent and popular events dominate access patterns.

9 Batch 1 Gap Cross-Reference and Consolidated Risk Assessment

The following table maps each Batch 1 gap to its impact on Batch 2 flows and current remediation status:

Batch 1 Gap	Batch 2 Impact	Severity Escalation	Status
State machine in app code (not Step Functions)	Event Creation wizard and RSVP flow lack orchestration resilience, no visual debugging, manual retry logic	Elevated — payment and capacity flows need saga support [15, 13]	Unremediated
JWT tokens lacking rotation	Payment endpoints vulnerable to token replay; non-compliant with RFC 9700	Critical — PCI DSS context [6, 7]	Unremediated
No DLQs on SQS	Failed payment webhooks, RSVP confirmations, and refund messages are silently lost	Critical — financial data loss risk [15, 22, 16]	Unremediated
Missing WAF	Payment API endpoints exposed to injection and XSS attacks	Critical — PCI DSS 4.0 requirement [3, 26]	Unremediated
Missing GuardDuty	No threat detection on Lambda invocations processing payments	Critical — no anomaly detection [4, 5]	Unremediated
No EventBridge event bus	Tight coupling between services; cancellation cascades require point-to-point calls	Elevated — scalability and reliability [47, 48]	Unremediated

Table: Batch 1 Gap Cross-Reference — Impact on Batch 2 Event Management Flows

All five Batch 1 gaps remain unremediated and have escalated in severity due to the payment processing and concurrency requirements of the Event Creation and RSVP flows. The three security gaps (WAF, GuardDuty, JWT rotation) are now P0-Critical given PCI DSS v4.0.1 mandatory compliance.

10 Prioritized Remediation Roadmap

Chart Explanation: This chart presents all remediation items grouped by priority level (P0-Critical through P3-Low) with estimated engineering effort in days. P0 items (WAF, GuardDuty, DLQ configuration, JWT rotation) total approximately 12 engineering days and address Batch 1 carry-forward gaps that are now blocking payment security compliance. P1 items (Step Functions migration, RSVP concurrency control, EventBridge bus) represent the largest effort at approximately 33 days but deliver the most architectural improvement. P2 and P3 items are incremental optimizations.

P0 — Critical (Batch 1 Carry-Forward, Target: Immediate). WAF and GuardDuty deployment (approximately 5 days) should be the first remediation, as these are table-stakes for any application processing payments through API Gateway [3, 4]. DLQ configuration across all SQS queues (approximately 3 days) prevents silent message loss for payment webhooks and RSVP confirmations [15, 22]. JWT token rotation implementation (approximately 4 days) with 15-minute access token lifetimes and per-use refresh token rotation addresses the RFC 9700 mandate [6, 7].

P1 — High (Architectural Improvements, Target: Next Sprint Cycle). Step Functions migration for both

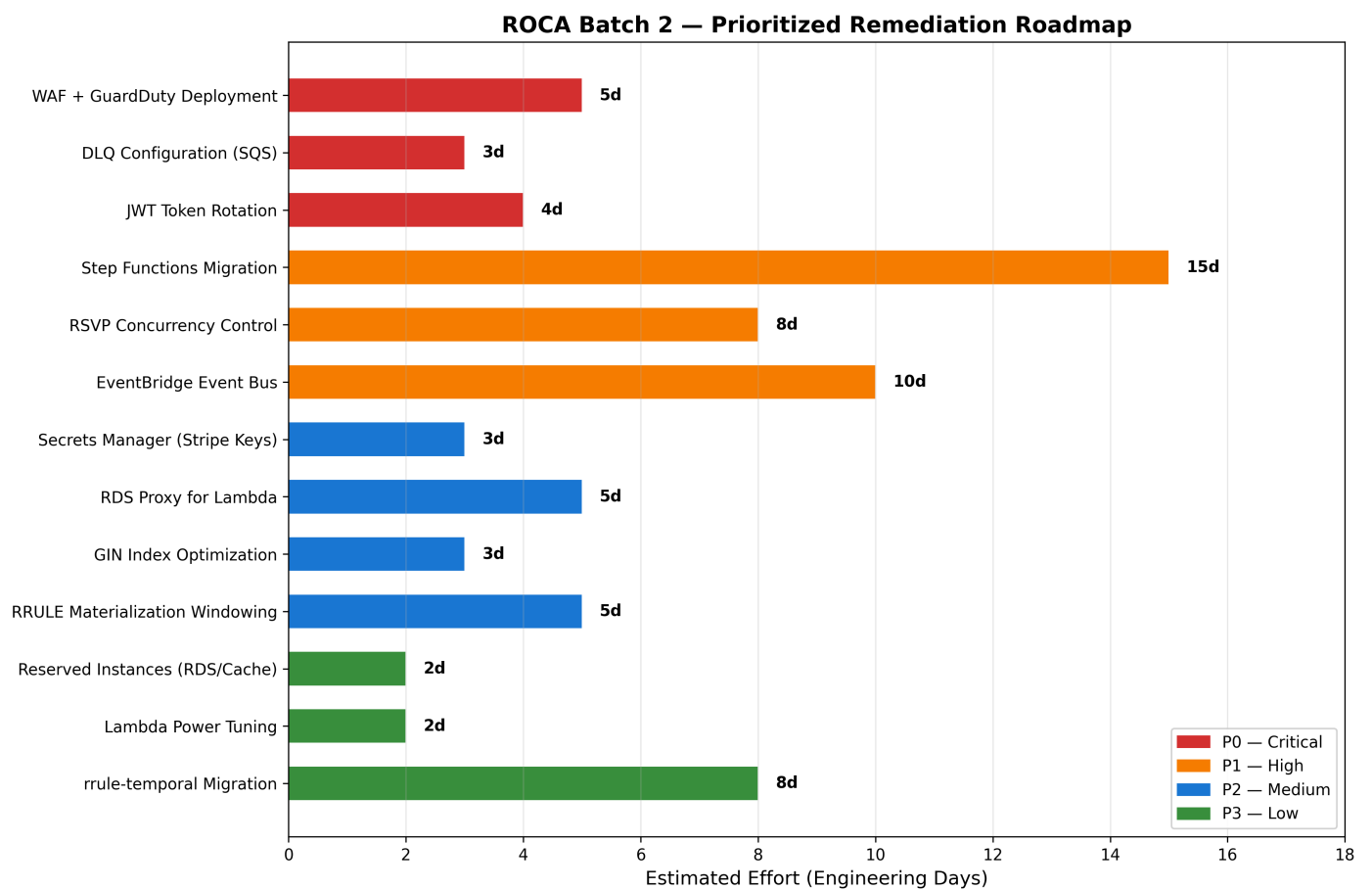


Figure 2: Remediation Roadmap

the Event Creation wizard and the RSVP/cancellation saga (approximately 15 days) is the largest single effort but delivers orchestration resilience, visual debugging, and built-in retry across all multi-step workflows [15, 13, 19]. RSVP concurrency control implementation (approximately 8 days) using SELECT FOR UPDATE SKIP LOCKED with atomic capacity decrements and Redis pre-locks addresses the highest-reliability risk [8, 9, 10]. EventBridge custom event bus deployment (approximately 10 days) decouples event lifecycle notifications, payment webhooks, and RSVP state changes into independently scalable consumers [47, 48].

P2 — Medium (Optimization, Target: Next Quarter). Secrets Manager rotation for Stripe API keys (approximately 3 days) leveraging Stripe's official AWS integration guide [30, 31]. RDS Proxy deployment for Lambda connection pooling (approximately 5 days) to prevent connection exhaustion during RSVP traffic spikes [11]. GIN index optimization on event search tsvector columns (approximately 3 days) with Redis cache-aside pattern [36, 38]. RRULE materialization windowing (approximately 5 days) to limit instance generation to 6–12 months ahead [17, 40].

P3 — Low (Efficiency Gains, Target: Ongoing). Reserved Instance commitments for RDS and ElastiCache (approximately 2 days of analysis and procurement) [43]. Lambda Power Tuning across all functions (approximately 2 days) [44, 45]. Migration from rrule.js to rrule-temporal for improved DST handling (approximately 8 days) [39, 40].

The total estimated remediation effort across all priorities is approximately 73 engineering days, with the P0-Critical items requiring only about 12 days and delivering the most immediate risk reduction. ROCA should execute P0 items in the current sprint, plan P1 items for the next sprint cycle, and schedule P2 and P3 items across the following quarter.

11 References

1. <https://compyl.com/guides/pci-dss-4-0-1-compliance-guide/>
2. <https://blog.pcisecuritystandards.org/coffee-with-the-council-podcast-guidance-for-pci-dss-e-commerce-requirements-effective-after-31-march-2025>
3. <https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-control-access-aws-waf.html>
4. <https://docs.aws.amazon.com/guardduty/latest/ug/lambda-protection.html>
5. <https://aws.amazon.com/it/blogs/security/how-you-can-use-amazon-guardduty-to-detect-suspicious-activity-within-your-aws-account/>
6. <https://ory.sh/blog/oauth-token-lifecycle-management>
7. <https://arxiv.org/html/2608.04115v1>
8. <https://www.cybertec-postgresql.com/en/skip-locked-one-of-my-favorite-9-5-features/>
9. <https://www.postgresql.org/docs/13/explicit-locking.html>
10. <https://redis.io/docs/latest/develop/clients/patterns/distributed-locks/>
11. <https://aws.amazon.com/it/blogs/compute/using-amazon-rds-proxy-with-aws-lambda/>
12. https://stripe.com/docs/api/idempotent_requests
13. <https://docs.aws.amazon.com/prescriptive-guidance/latest/patterns/implement-the-serverless-saga-pattern-by-using-aws-step-functions.html>
14. <https://us-east-1.quicksight.aws.amazon.com/sn/spaces/8f018b40-d0a9-42f1-a7d5-e0503a653d43?documentId=1105e813-4bbb-b559-e22a1eb0358a>
15. <https://docs.aws.amazon.com/wellarchitected/latest/serverless-applications-lens/failure-management.html>
16. <https://aws.amazon.com/sqs/faqs/>
17. <https://us-east-1.quicksight.aws.amazon.com/sn/spaces/f756a661-a188-4112-bed6-e70567713dbc?documentId=c7258398-fa22-4df7-8c1c-ecf307ce2a78>
18. <https://www.serverless.com/guides/aws-step-functions>
19. <https://docs.aws.amazon.com/step-functions/latest/dg/concepts-error-handling.html>
20. <https://docs.aws.amazon.com/solutions/latest/account-assessment-for-aws-organizations/aws-well-architected.html>
21. <https://aws.amazon.com/blogs/opensource/simplifying-serverless-best-practices-with-lambda-powertools/>

22. <https://us-east-1.quicksight.aws.amazon.com/sn/spaces/8f018b40-d0a9-42f1-a7d5-e0503a653d43?documentId=7407330c-4032-b8ca-f4ed92956eda>
23. <https://blog.getagency.com/articles/pci-compliance-cloud-saas>
24. <https://cside.com/blog/does-stripe-make-you-pci-compliant-6-4-3-11-6-1>
25. <https://blog.report-uri.com/stripe-now-asks-you-to-attest-that-youve-deployed-a-csp/>
26. <https://aws.amazon.com/about-aws/whats-new/2025/03/aws-waf-pci-compliance-protection-partner-solutions/>
27. <https://duendesoftware.com/learn/best-practices-managing-token-expiration-refresh-revocation-in-web-apis>
28. <https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-jwt-authorizer.html>
29. <https://datatracker.ietf.org/doc/html/rfc10017>
30. <https://docs.aws.amazon.com/secretsmanager/latest/userguide/api-keys-security-sensitive.html>
31. <https://stripe.dev/blog/securing-stripe-api-keys-aws-automatic-rotation>
32. Source: wikipedia.org
33. <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/rds-proxy-pinning.html>
34. <https://redis.antirez.com/fundamental/redlock.html>
35. <https://docs.stripe.com/batch-api/best-practices>
36. <https://www.postgresql.org/docs/current/textsearch-indexes.html>
37. https://aws.amazon.com/blogs/database/index-types-supported-in-amazon-aurora-postgresql-and-amazon-rds-for-postgresql-using-extensions-sp-gist-btree_gin-and-btree_gist/
38. <https://redis.io/tutorials/howtos/solutions/microservices/caching/>
39. <https://github.com/jkbrzt/rrule/blob/master/README.md>
40. <https://github.com/ggaabe/rrule-temporal>
41. <https://docs.aws.amazon.com/lambda/latest/dg/best-practices.html>
42. <https://docs.aws.amazon.com/lambda/latest/dg/configuration-concurrency.html>
43. <https://docs.aws.amazon.com/whitepapers/latest/how-aws-pricing-works/aws-cost-optimization.html>
44. <https://docs.aws.amazon.com/lambda/latest/dg/configuration-memory.html>
45. <https://aws.amazon.com/blogs/architecture/optimizing-your-aws-infrastructure-for-sustainability-part-i-compute/>
46. <https://aws.amazon.com/it/blogs/compute/efficiently-processing-batched-data-using-parallelization-in-aws-lambda/>
47. <https://docs.aws.amazon.com/prescriptive-guidance/latest/modernization-integrating-microservices/event-bridge.html>
48. <https://aws.amazon.com/blogs/compute/introducing-amazon-eventbridge-schema-registry-and-discovery-in-preview/>